

1 The 1D Lunar Soft Landing Problem

In this first article on lunar soft landing, I cover the theoretical side of building a numerical algorithm to find the optimal engine ignition time – ensuring a soft landing while minimizing fuel consumption.

1.1 Formulation of the 1D Lunar Soft Landing Problem

Let's consider a spacecraft in the process of landing. Its state at a specific time is described by a vector $[m(t), v(t), h(t), t]$, where:

- $m(t)$ is the mass of the spacecraft,
- $v(t)$ is its vertical velocity,
- $h(t)$ is its altitude.

The spacecraft starts its landing at altitude h_0 with velocity v_0 and mass m_0 (state vector $[m_0, v_0, h_0, 0]$).

The lunar gravitational acceleration $g = 1.622 \text{ m/s}^2$ acts on it. The spacecraft has a fuel reserve $m_{\text{fuel}} < m_0$. During engine operation, fuel is burned with a rate coefficient k , and the thrust satisfies $0 \leq u(t) \leq u_{\text{max}}$, where $u(t)$ is an unknown control function.

The landing process is described by the following system of differential equations:

$$\begin{cases} \frac{dh}{dt} = v(t) \\ \frac{dv}{dt} = -g + \frac{u(t)}{m(t)} \\ \frac{dm}{dt} = -k \cdot u(t) \end{cases}$$

The goal is to find a control $u(t)$ such that, at the final time τ , the conditions $v(\tau) = 0$ and $h(\tau) = 0$ are met, and the fuel consumption is minimized.

1.2 The Form of the Optimal Control Function

I'll skip the detailed mathematical derivation of the control function. If you want, you can find it in the optimal control literature. The most important thing we need to know is that, according to Pontryagin's maximum principle, the solution is a bang-bang control:

1. The spacecraft is in free fall before the switching time t_s ($u(t) = 0$).
2. The engine of the spacecraft operates at maximum thrust ($u(t) = u_{\text{max}}$) after time t_s .

Does this mean the goal is achieved? But in fact, it is only the beginning, because the most important problem is to find the unknown switching time t_s .

1.3 System description for the two landing stages

The landing process is split into two stages, in accordance with the nature of optimal control:

1. During the free-fall period ($0 \leq t \leq t_s$), the change of state is described by the following system of ODEs (with $u(t) = 0$):

$$\begin{cases} \frac{dh}{dt} = v(t) \\ \frac{dv}{dt} = -g \\ \frac{dm}{dt} = 0 \end{cases}$$

With the initial conditions $h(0) = h_0$, $v(0) = v_0$, $m(0) = m_0$.

This represents a Cauchy problem. Solving this problem yields explicit solutions that directly describe the state of the spacecraft at any time during free fall:

$$\begin{cases} h_{\text{free}}(t) = h_0 + v_0 t - \frac{gt^2}{2} \\ v_{\text{free}}(t) = v_0 - gt \\ m_{\text{free}}(t) = m_0 \end{cases}$$

2. During the braking stage ($t_s \leq t \leq \tau$), the change of state takes the following form:

$$\begin{cases} \frac{dh}{dt} = v(t) \\ \frac{dv}{dt} = -g + \frac{u_{\text{max}}}{m(t)} \\ \frac{dm}{dt} = -k \cdot u_{\text{max}} \end{cases}$$

The left boundary conditions correspond to the state of the spacecraft at time t_s :

$$(h_{\text{free}}(t_s), v_{\text{free}}(t_s), m_{\text{free}}(t_s)).$$

The right boundary conditions represent the soft landing requirements:

$$h(\tau) = 0, v(\tau) = 0.$$

What are the main challenges?

We have a boundary value problem (BVP), where the switching time t_s and the final time τ are unknown in advance. The switching time t_s directly sets the left boundary conditions for the second stage, and therefore it determines the trajectory and the final landing time.

1.4 The Shooting Method for Solving the BVP

Now we come to the most interesting part: *How can we determine the optimal time t_s for engine activation and construct the corresponding spacecraft trajectory? Which numerical method can help us solve this BVP?*

The Shooting Method is well-suited for this purpose. This method reduces a complex BVP to finding the root of a residual function by integrating the initial value problem (Cauchy problem) at each iteration (shot) and modifying a free parameter of the system (shooting parameter).

Let's put it into algorithmic terms. Instead of using an abstract theoretical construction, we build an algorithm that runs a series of landing simulations (e.g., via 4th-order Runge-Kutta/RK4):

1. **Shooting parameter:** We change the engine activation time t_s .
2. **Shot:** For each chosen time t_s , we run a simulation of the second stage and integrate the system step-by-step until the stopping condition is met (*until the velocity v reaches 0*).
3. **Computing the residual:** At the end of the simulation, we measure the final altitude h .

The purpose of the algorithm is to modify the engine activation time t_s in a series of simulations until the final altitude h equals zero at the final time τ (*the soft landing condition*).

Let's return to the mathematical component. There are two primary formulations that serve as the basis for constructing the residual function $F(t_s)$. To illustrate this, let us consider the conditions for a soft landing: at the moment of landing, both target altitude and velocity must be equal to zero ($h(\tau) = h_\tau = 0$ and $v(\tau) = v_\tau = 0$).

What does this mean in practice? We can run a simulation with a chosen value of t_s up to one of two terminal stopping conditions:

- **Terminal Altitude Function** $H(t_s, \tau)$: Integrate until $v(t_s, \tau) = v_\tau$ (a full stop) and evaluate the final altitude $H(t_s, \tau) = h(t_s, \tau)|_{v(t_s, \tau)=v_\tau}$. The residual function is $F(t_s) = H(t_s, \tau) - h_\tau$.
- **Terminal Velocity Function** $V(t_s, \tau)$: Integrate until $h(t_s, \tau) = h_\tau$ (surface contact) and evaluate the final velocity $V(t_s, \tau) = v(t_s, \tau)|_{h(t_s, \tau)=h_\tau}$. The residual function is $F(t_s) = V(t_s, \tau) - v_\tau$.

For a nominal soft landing ($h_\tau = 0, v_\tau = 0$), using the terminal altitude and terminal velocity functions simplifies the residual to $F(t_s) = H(t_s, \tau)$ or $F(t_s) = V(t_s, \tau)$.

In other words, two functions (*the final velocity function upon surface touchdown and the final altitude function upon a full stop*), on paper, can act as the residual function $F(t_s)$. But an engineering question arises: Are both options suitable for numerical algorithms?

1.5 Analysis of Final Velocity and Altitude Plots

To answer this question, we run a series of simulations for a sequence of t_s values and plot the graphs:

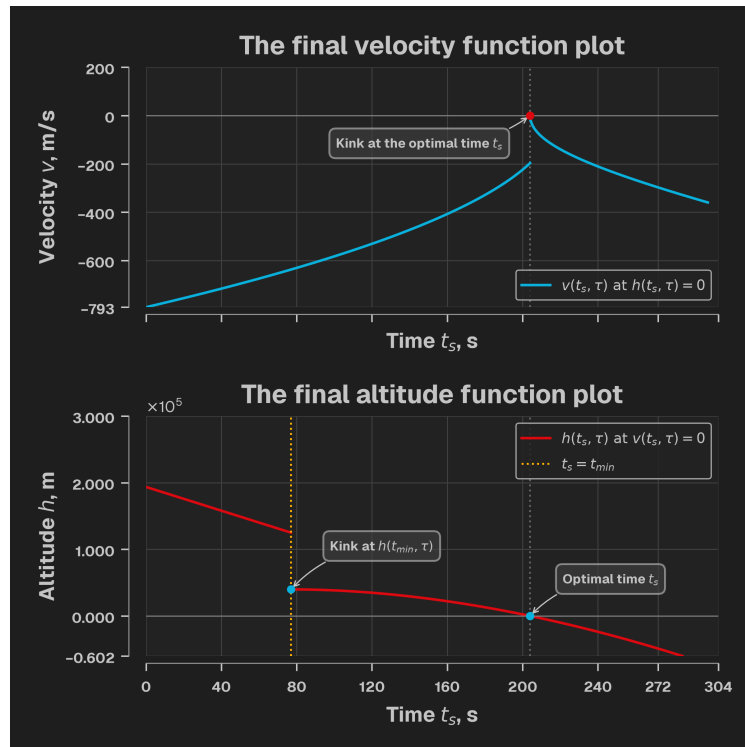


Figure 1: The final velocity and altitudes plots

The first plot shows the dependence of the final velocity on the engine activation time. It has a kink at the optimal time for activating the engine. This is a dealbreaker for us: we cannot use methods with high-order convergence to find the root of the final velocity function, because these methods will diverge at the optimal time t_s .

In the second plot (*the final altitude function*), we observe a kink, but this time at the point $t_s = t_{\min}$. This kink is due to the physics of landing. If we run a simulation on the interval $[0, t_{\min})$, our spacecraft will ascend rather than brake, because its initial velocity is oriented upward ($v_{\text{free}} > 0$). Accordingly, we can omit this interval – it makes no physical sense for the soft landing problem.

You might expect that further on the interval $[t_{\min}, +\infty)$, the final altitude function would definitely be smooth and continuous. However, in practice, the domain of the function is bounded by $t_{\max}$.

In other words, we can't push back the engine activation time t_s to infinity and always obtain a final altitude value. What is the reason? Let us recall the stopping condition of the simulation: our loop iterates until the spacecraft completely cancels out its free-fall velocity.

However, if the spacecraft starts braking and lacks sufficient thrust or velocity reserve (Δv), then the spacecraft's velocity does not cross zero. Unless a program interlock is implemented, the simulation enters an infinite loop. Mathematically, it means that for $t_s > t_{\max}$, the value of the function $H(t_s, \tau)$ does not exist. I will cover the details of the program interlock in a subsequent article.

The crucial point is that the final altitude function is smooth on the closed interval $[t_{\min}, t_{\max}]$. Accordingly, we can deploy the full arsenal of numerical methods for root finding without

difficulty.

1.6 Determining the Bounds of the Target Interval: $t_{\min}$ and $t_{\max}$

Now, a practical question arises: How can we precisely determine the feasible interval $[t_{\min}, t_{\max}]$? After all, we cannot act blindly when choosing values for the simulation.

1. Finding the lower bound $t_{\min}$

Recall the physical meaning of the time $t_{\min}$. It is the time when the vertical velocity of the spacecraft during free-fall equals zero: $v_{\text{free}}(t_{\min}) = 0$. Before this moment, the spacecraft is gaining altitude ($v_{\text{free}} > 0$); after this moment, it begins descending ($v_{\text{free}} < 0$).

To find the time $t_{\min}$, let us apply this condition to the equation $v_{\text{free}}(t) = v_0 - gt$. From this, we obtain:

$$t_{\min} = \frac{v_0}{g}$$

Nonetheless, what do we do if the initial velocity v_0 is negative (from the outset, the spacecraft is descending)? We will not “go back in time.” Therefore, we adjust the formula for $t_{\min}$ accordingly:

$$t_{\min} = \begin{cases} 0, & \text{if } v_0 \leq 0 \\ \frac{v_0}{g}, & \text{if } v_0 > 0 \end{cases}$$

2. Finding the upper bound $t_{\max}$

Now let us turn to $t_{\max}$. This is the theoretical time limit for engine ignition at which the spacecraft can still cancel its descent velocity.

From the equation $v_{\text{free}}(t_{\max}) - \Delta v = 0$ (where Δv is the available velocity reserve), we obtain the following formula:

$$t_{\max} = \frac{v_0 - \Delta v}{g}$$

(It should be noted that these formulas are precise under constant gravitational acceleration g . In the case of dynamic $g(h)$, they provide only a preliminary approximation.)

The Δv reserve is determined by the results of a test simulation (if required, we ignore the surface and allow the spacecraft to pass through it). As a starting point of the simulation, we take the time of impact of the spacecraft resulting from its free fall. This gives us the following:

1. If the spacecraft has sufficient Δv , then the upper bound $t_{\max}$ becomes the impact time.

2. Otherwise, we compute the difference between the final and initial velocities when the fuel is exhausted. This difference serves as our Δv for computing the new $t_{\max}$.

The impact time is derived from the equation $h_{\text{free}}(t) = h_0 + v_0 t - \frac{gt^2}{2}$ and has the following form:

$$t_{\max} = \frac{v_0 + \sqrt{v_0^2 + 2gh_0}}{g}$$

1.7 Halley's Method for Finding the Root of the Final Altitude Function

When conducting a series of simulations, we cannot afford to brute-force the available engine ignition time from the interval $[t_{\min}, t_{\max}]$. We need a reliable mathematical guide - an algorithm that predicts the next step $t_{s,n+1}$ so as to minimize the time required to reach the optimal value. For this reason, we apply Halley's method.

Why this method rather than the classical Newton's method? The main reason is that Halley's method has a cubic order of convergence. Since the computation of the residual function $F(t_s)$ (the final altitude function) does not consist of simple number substitution, but rather requires running a complete landing simulation, our task is to minimize the number of runs. The higher order of convergence allows us to approximate the root in fewer iterations.

In Halley's method, a more precise step t_s is calculated using the following formula:

$$t_{s,n+1} = t_{s,n} - \frac{2F(t_{s,n})F'(t_{s,n})}{2[F'(t_{s,n})]^2 - F(t_{s,n})F''(t_{s,n})}$$

The required derivatives can be computed numerically using central finite differences:

$$F'(t_{s,n}) \approx \frac{F(t_{s,n} + \varepsilon) - F(t_{s,n} - \varepsilon)}{2\varepsilon}$$

$$F''(t_{s,n}) \approx \frac{F(t_{s,n} + \varepsilon) - 2F(t_{s,n}) + F(t_{s,n} - \varepsilon)}{\varepsilon^2}$$

(While higher-order methods typically demand computing complex second derivatives, our implementation circumvents this cost. By applying a central finite-difference scheme on the three-point stencil $\{t_s - \varepsilon, t_s, t_s + \varepsilon\}$, both $F'(t_s)$ and $F''(t_s)$ are derived simultaneously using only three simulation runs per iteration. Consequently, Halley's method achieves cubic convergence with virtually the same computational cost per step as Newton's method using central differences.)

1.8 Numerical Algorithm for Finding the Optimal Engine Ignition Time

At this stage, let us compile all elements into a single sequential framework. Here is how the algorithm works step-by-step:

Step 1: Localization of initial boundaries

Compute the values $t_{\min}$ and $t_{\max}$ (the impact time).

Step 2: Checking and correcting the time $t_{\max}$

Run a test simulation for the impact time $t_{\max}$ and check the existence of the value of the residual function.

2.1. If the value of the residual function does not exist: Compute the available Δv and correct the right boundary via the following formula: $t_{\max} = \frac{v_0 - \Delta v}{g}$.

2.2. Validation of the new $t_{\max}$: Start a simulation for the updated time $t_{\max}$. If $F(t_s) > 0$ (the final altitude is greater than zero), then a solution does not exist (the spacecraft's thrust or fuel is insufficient for a soft landing). If $F(t_s) \leq 0$, then choose $t_{\max}$ as the initial approximation for Halley's method: $t_0 = t_{\max}$.

Step 3: Iteration loop (Halley's method)

Initiate a loop where each iteration runs a simulation and performs other required computations:

3.1. Computing the next approximation: Compute the new time t_{i+1} via Halley's formula.

3.2. Clamping: Check whether t_{i+1} belongs to the interval $[t_{\min}, t_{\max}]$.

- If $t_{i+1} < t_{\min}$, then $t_{i+1} = t_{\min}$.
- If $t_{i+1} > t_{\max}$, then $t_{i+1} = t_{\max}$.

(Important nuance. If t_{i+1} is corrected, then the relevant boundary needs to be shifted inside the interval by a small step dt so as to prevent getting stuck in an infinite loop.)

3.3. Verifying the stopping criterion: Compute the value of the residual function $F(t_i)$. If $|F(t_i)| < \epsilon$ (ϵ is a given tolerance, e.g., $\epsilon = 10^{-5}$ m), then stop the loop and return t_i as the optimal time t_s and the corresponding trajectory of the spacecraft.

1.9 Examples

Now, let us examine the numerical results generated by the solver. We will analyze three distinct descent scenarios with varying initial conditions and spacecraft configurations to demonstrate the program's capabilities.

1. Standard Low Lunar Orbit (LLO) Descent (TWR 1.42)

In this baseline scenario, we define the spacecraft's initial state with a mass of 28,877 kg, a velocity of -56 m/s, and an altitude of 102,378 m. The target boundary condition for a soft landing is strictly set to zero velocity and zero altitude. The propulsion system provides a maximum thrust of 66,628 N, with a fuel mass capacity of 17,644 kg.

Flight Dynamics: The optimal engine ignition time (t_s) was calculated at 178.87 seconds

into the flight, occurring at an altitude of 66,390.54 m.

Outcome: The active braking phase successfully neutralized a free-fall velocity of -346.38 m/s, resulting in a safe touchdown at 521.97 seconds with 22,041.01 kg of vehicle mass remaining.

Below, you can see the generated phase trajectory and telemetry plots for this standard descent profile.

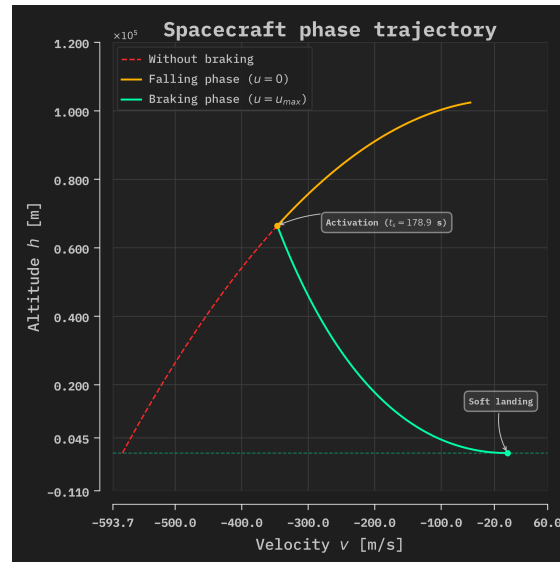


Figure 2: Phase trajectory

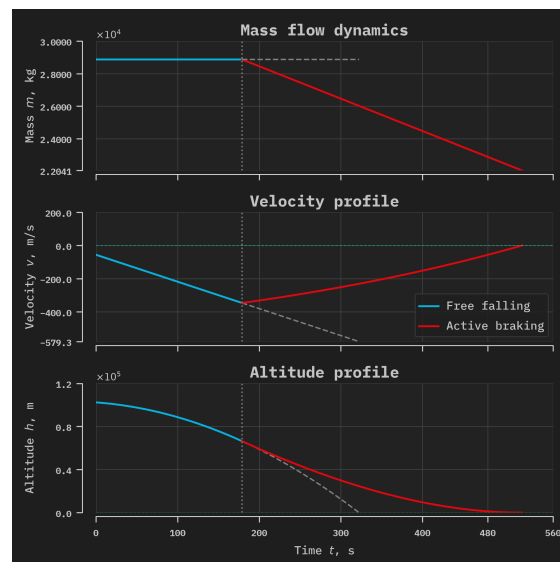


Figure 3: Spacecraft's telemetry

2. Heavy Cargo Landing (TWR 1.04, Critical Mass)

This edge-case scenario tests the solver against a low Thrust-to-Weight Ratio. The initial kinematic state is set to a mass of 38,852 kg, a velocity of -109 m/s, and an altitude of

105,674 m. The spacecraft is equipped with a 65,848 N engine and 15,358 kg of available fuel.

Flight Dynamics: Due to the heavy cargo and lower TWR, the engine ignition was required much earlier, at 76.42 seconds at an altitude of 92,603.07 m.

Outcome: The system successfully countered a velocity of -233.07 m/s, achieving a terminal landing at 686.48 seconds with a final mass of 27,473.33 kg.

The phase trajectory and telemetry plots illustrating this high-inertia descent are presented below.

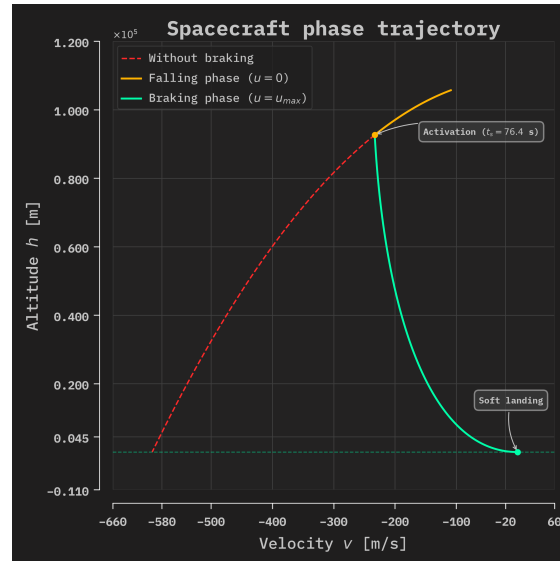


Figure 4: Phase trajectory

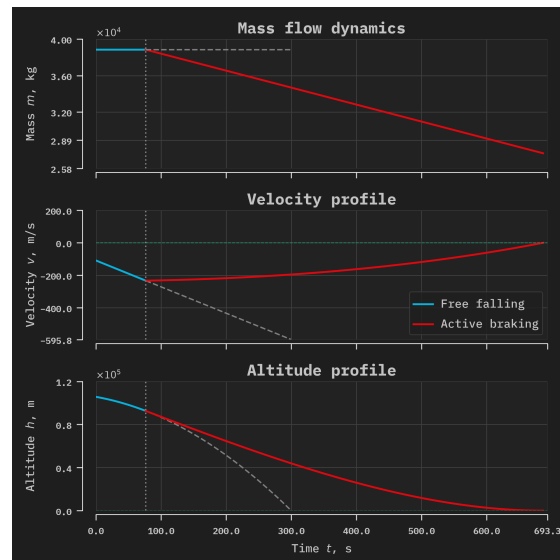


Figure 5: Spacecraft's telemetry

3. Orbital Drop (Zero Initial Velocity)

In the final scenario, we simulate a direct vertical drop. The initial conditions are characterized by a heavy mass of 46,535 kg, a near-zero initial velocity (4 m/s), and an altitude of 106,691 m. The rocket features a powerful 98,004 N engine and carries 27,900 kg of fuel.

Flight Dynamics: The optimal braking maneuver was initiated at 207.21 seconds at an altitude of 72,667.14 m.

Outcome: After neutralizing a peak free-fall velocity of -332.39 m/s, the spacecraft landed smoothly at 580.46 seconds, retaining 32,298.01 kg of its mass.

The resulting phase trajectory and residual function plots for this specific regime are shown below.

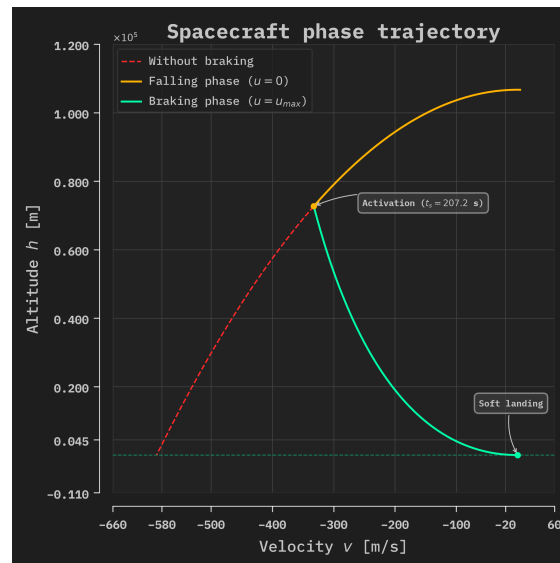


Figure 6: Phase trajectory

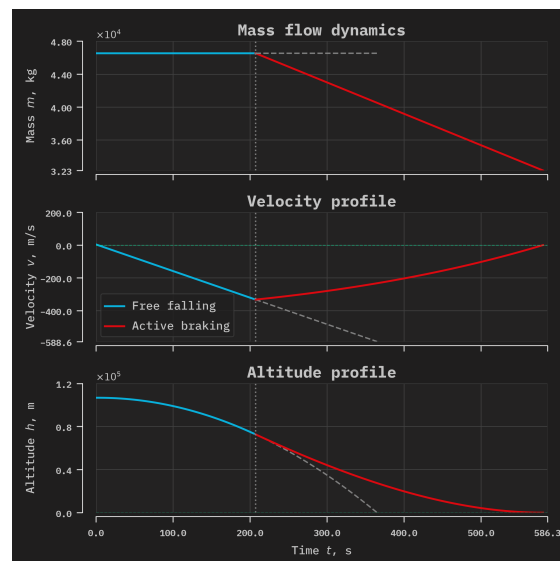


Figure 7: Spacecraft's telemetry